UNIVERSITY OF PADOVA
DEPARTMENT OF INFORMATION ENGINEERING

Biomedical Wearable Technologies
for Healthcare and Wellbeing

UX and UI in Flutter

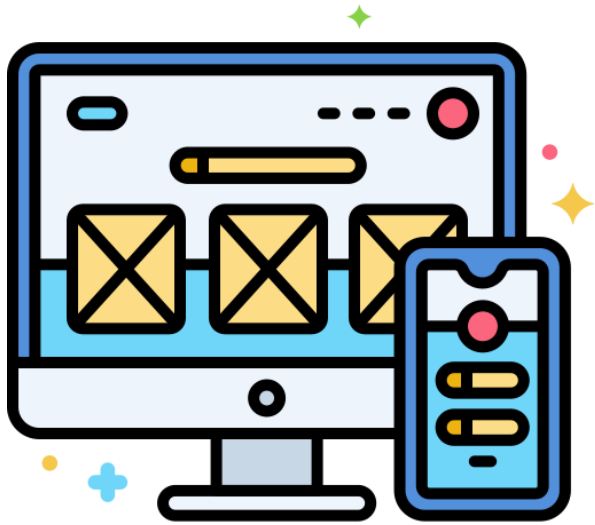
A.Y. 2025-2026

Giacomo Cappon

DEPARTMENT OF
INFORMATION
ENGINEERING
UNIVERSITY OF PADOVA

User eXperience (UX)

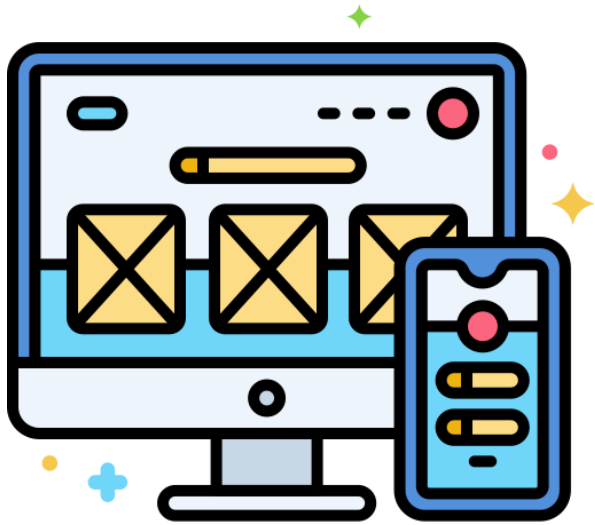
- **UX** (User eXperience) is the **total experience** a person has while interacting with a product.
- It includes perception, emotional response, efficiency, clarity, friction, trust.
- UX is not just screens. UX is the entire journey.
- When UX is ok? You should ask yourself:
  - Does the product solve a real problem?
  - Is the user goal clear?
  - Is the path to that goal obvious?
  - Does the system behave predictably?
  - Is error handling supportive?



User Interface (UI)

- **UI** (User Interface) is the **visual and interactive layer**.
- It includes typography, color, layout, spacing, components, animations,...
- What is the difference between UX and UI?

UI **communicates** structure.
UX **defines** structure.



Fundamental UX Principles

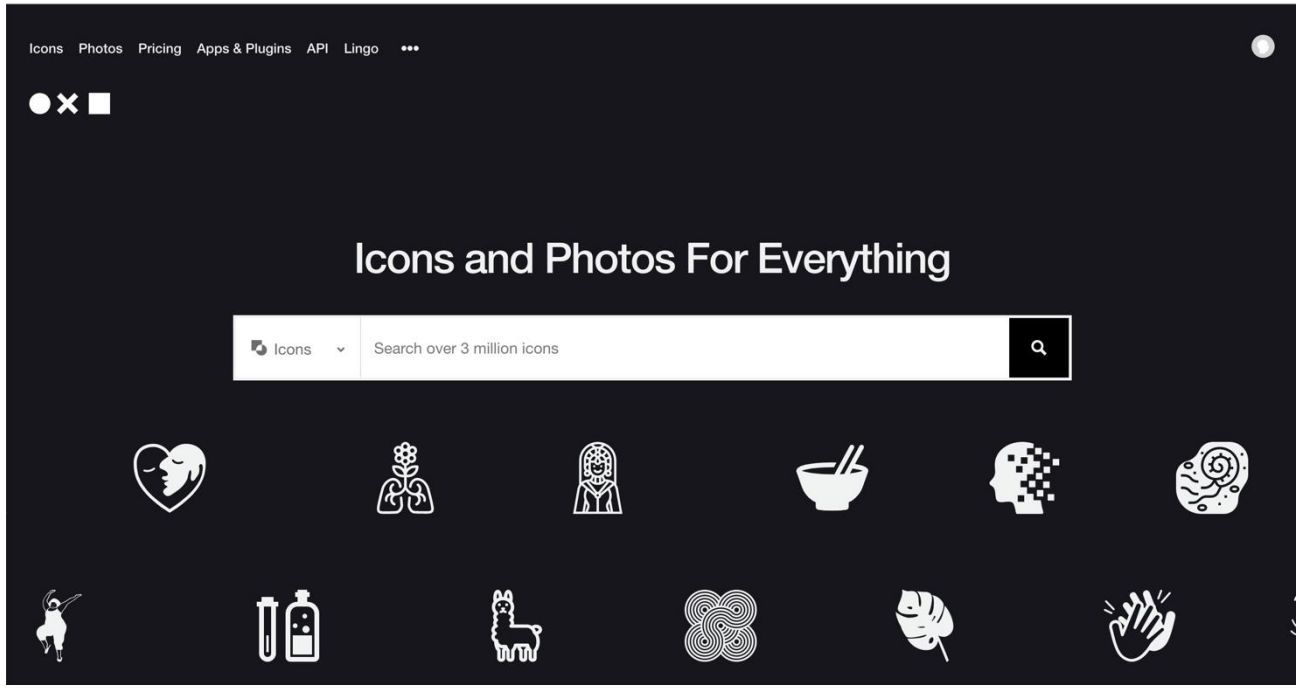
- UX grounds on **five** principles:
  - Visual Hierarchy
  - Consistency
  - Feedback
  - Cognitive Load
  - Accessibility
- These apply whether you build a website, mobile app, medical device, or car dashboard.

Visual Hierarchy

- Users scan. They do not read linearly.

- Hierarchy answers:
  - What matters most?
  - What is actionable?
  - What is secondary?

- Tools in our hands:
  - Setting the size of each component
  - Setting the contrast
  - Defining spacing
  - Setup and alignment policy
  - Grouping stuff together



- If everything is bold, nothing is bold.

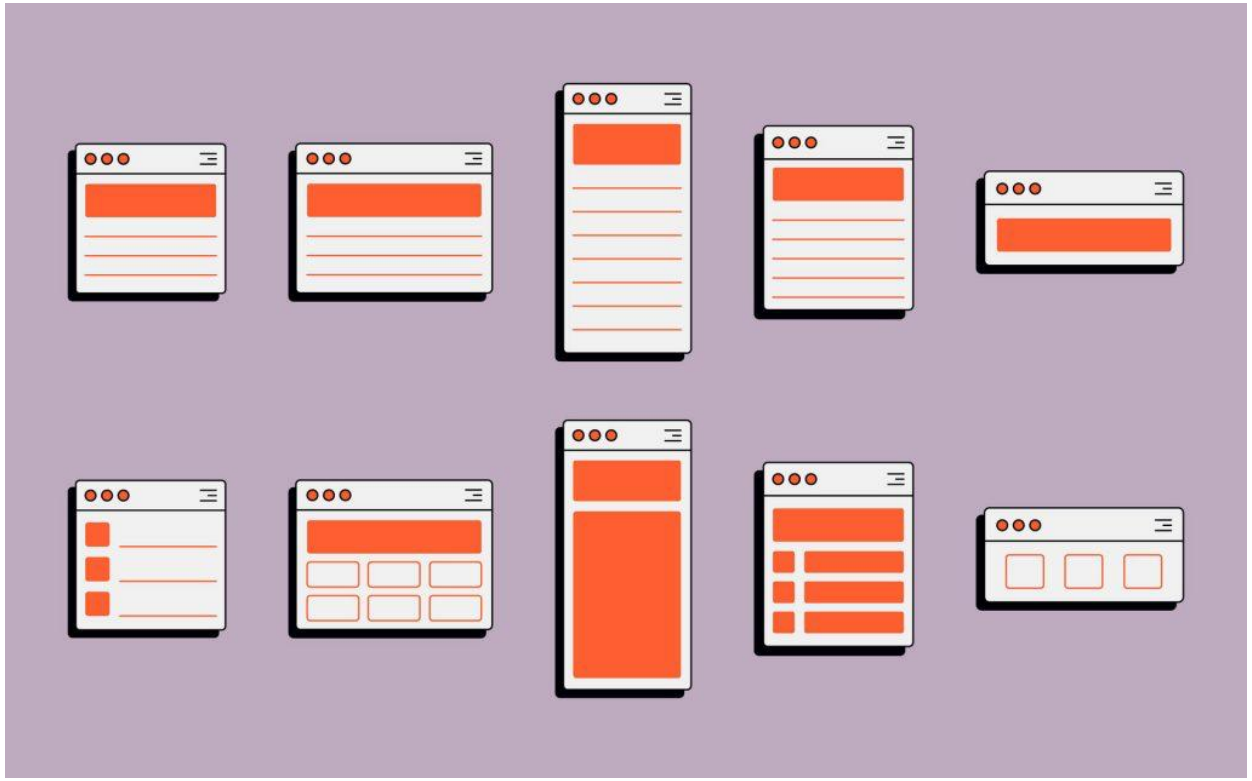
Consistency

- Humans recognize patterns.

- If
  - Buttons look different each time
  - Navigation changes unpredictably
  - Terminology varies

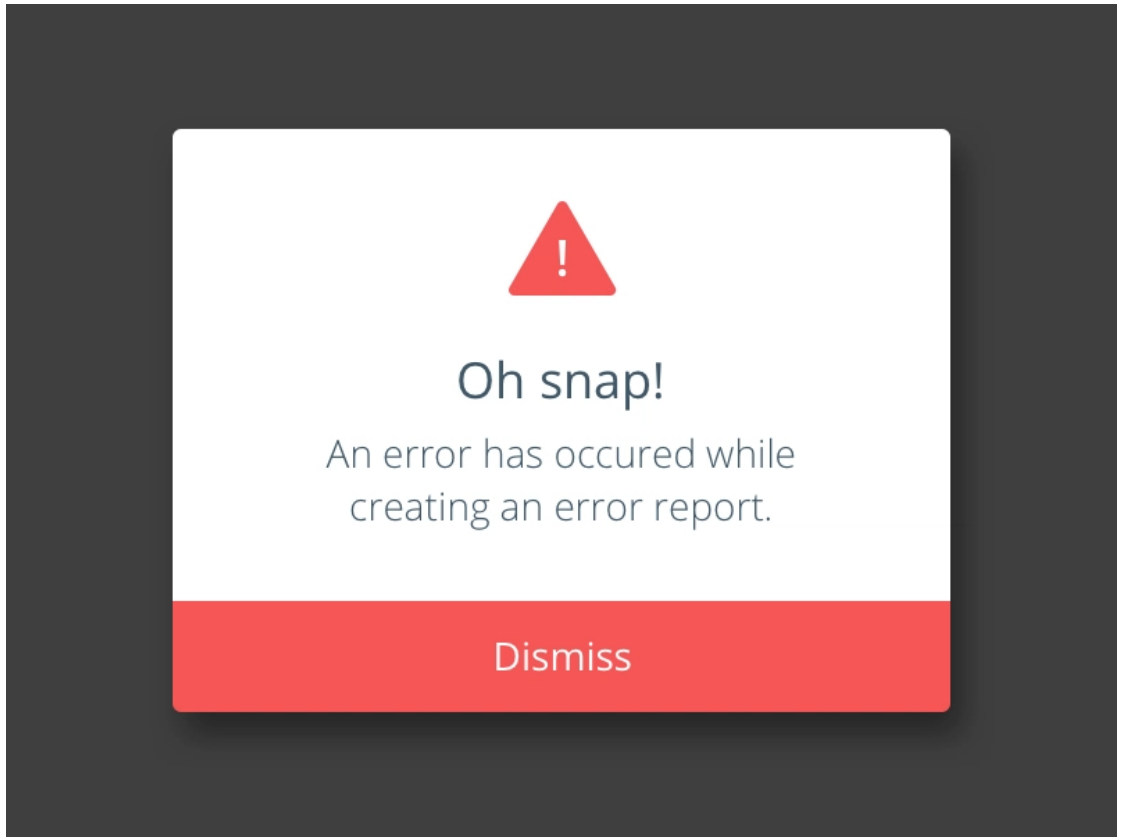
then, the cognitive load increases.

- Consistency builds confidence.



Feedback

- Every action must produce feedback.
- Examples:
  - Button press animation
  - Loading indicator
  - Error message
  - Success confirmation
- No feedback = anxiety. Did it worked?



Cognitive Load

- Cognitive load = Mental effort and stress

- Cognitive load increases when:
  - Too many options exist
  - Too much information is visible
  - Terminology is complex
  - Interaction patterns vary

- Goal: reduce thinking.

Accessibility

- Accessibility means:
  - Readable contrast
  - Scalable text
  - Clear semantics
  - Usable with assistive technologies
- Accessibility is not a feature. It is a quality standard.

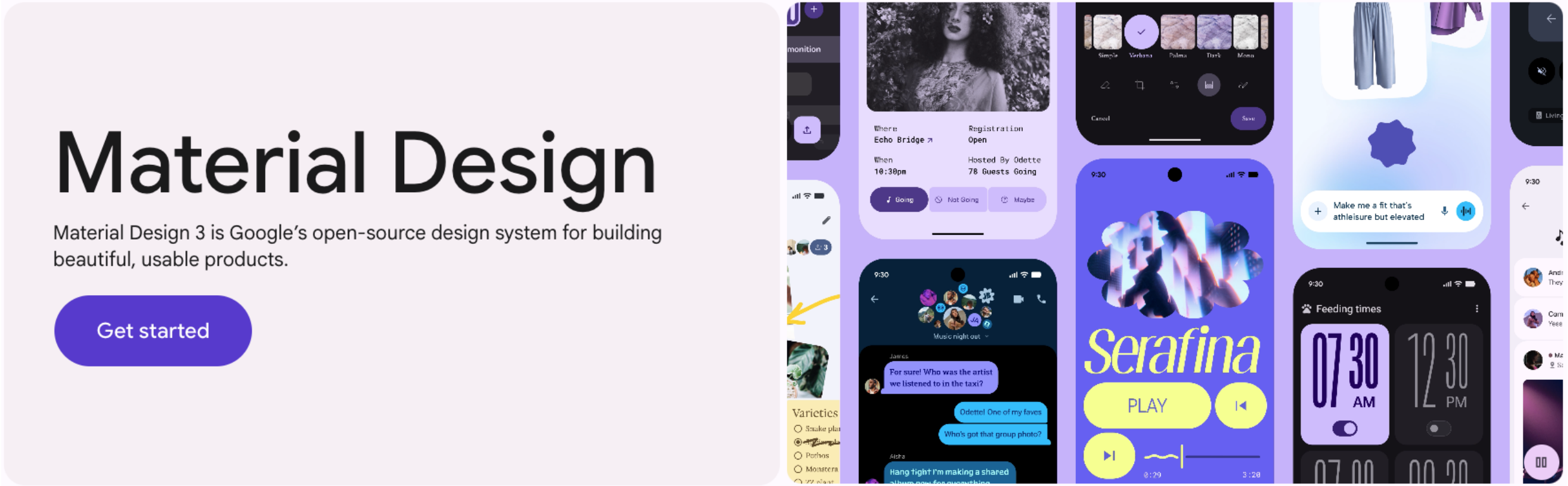
Design Systems

- A **design system** is a comprehensive, centralized repository of reusable components, standards, documentation, and design principles used to create consistent digital products at scale.

- It prevents:
  - Random design decisions
  - Visual inconsistency
  - Scaling problems

- Examples of design systems:
  - Apple Human Interface Guidelines
  - Microsoft Fluent
  - Material Design

Material Design



Material Design

- Material Design is not “Google style.”
- It is a **UX framework encoded into UI rules.**

- Core **ideas**:
  - *Structure through surfaces*: organizing UI into background, surfaces, and elevated components to create visual grouping.
  - *Meaningful motions*: explaining what changed, where content came from, and what happened.
  - *Clear hierarchy*: defining scales, button hierarchy, component roles.
  - *Color system*: using a primary, secondary, surface, error, contrasts

- This ensures accessible combinations.

Material Components as UX Decision

- Material components are structured by importance.
- **Let’s make some examples**
- All examples and best practices can be found at: https://m3.material.io/

Material Components as UX Decision

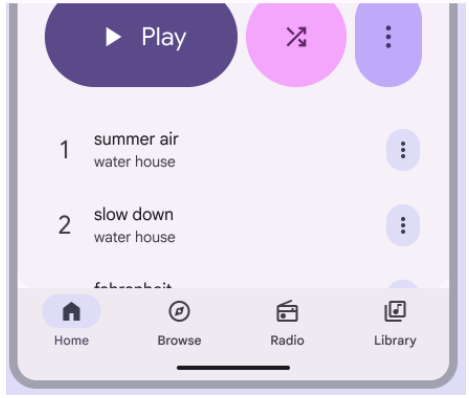
- **Buttons:**
  - Primary action → Elevated, Filled, and Tonal button
  - Secondary → Outlined button
  - Tertiary → Text button

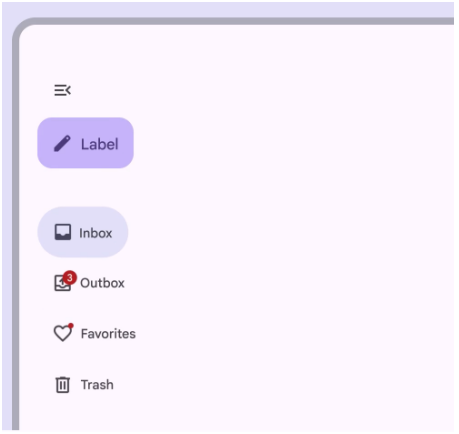
This encodes UX hierarchy visually.

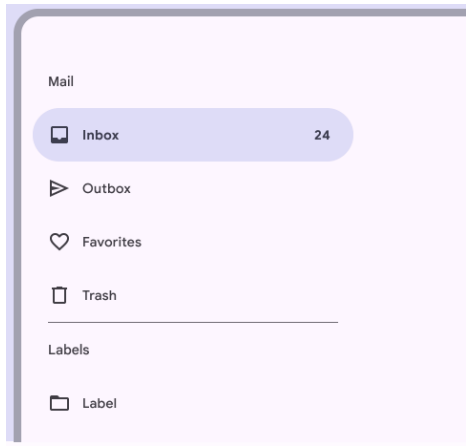
Material Components as UX Decision

- **Navigation patterns:**
  - Bottom navigation (few top sections)
  - Navigation rail (larger screens)
  - Drawer (complex systems)

- Material links navigation model to app size.



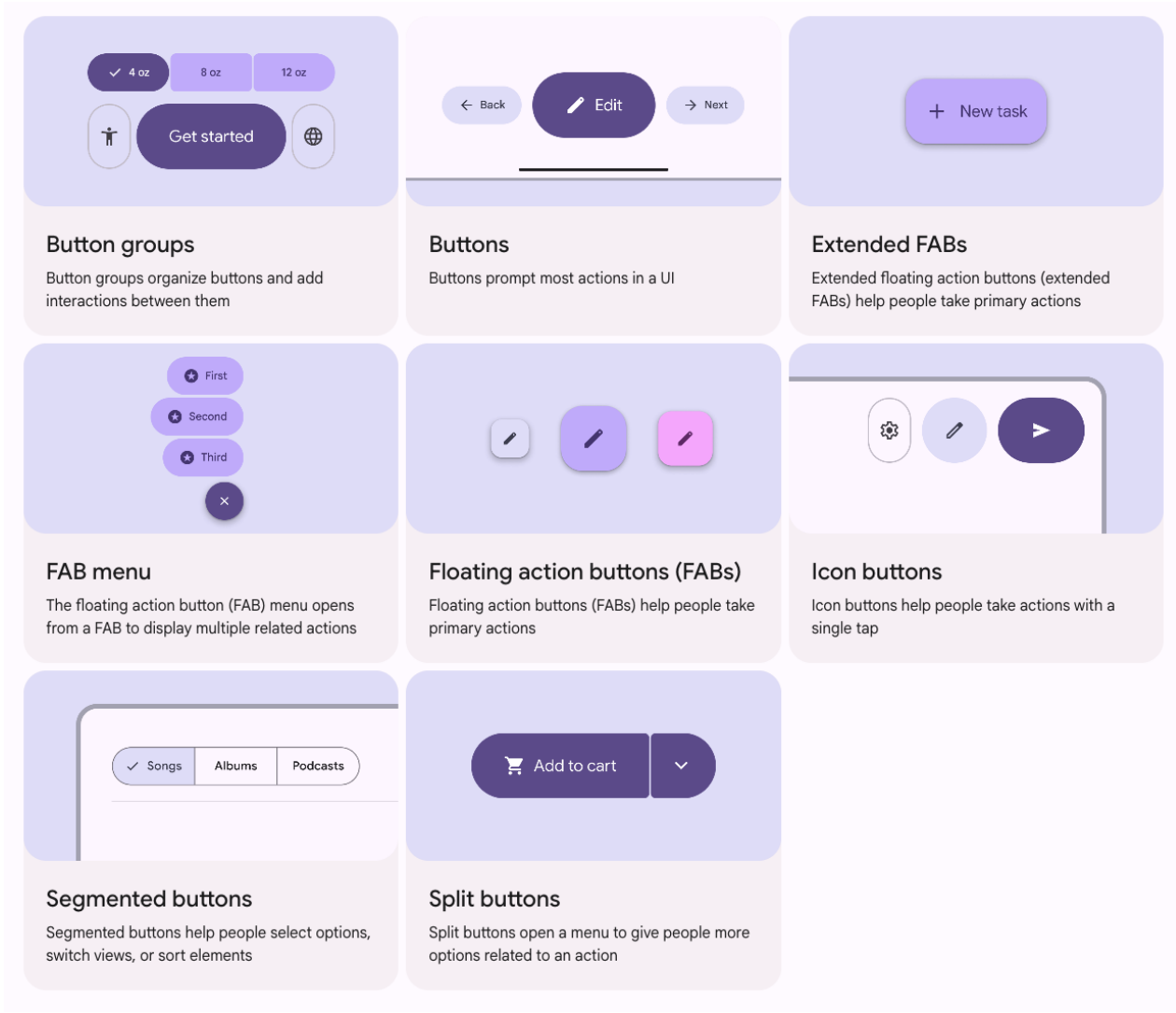




Material Components as UX Decision

- **Content organization:**
  - Cards for grouping
  - Lists for scanning
  - FAB for dominant actions

- Each has semantic meaning.



Activating Material in Flutter: Theming

- In Flutter, Material is activated providing a ThemeData to your MaterialApp through:

```
MaterialApp(
  theme: ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
  ),
)
```

- Why this matters:
  - Color harmony is automatic
  - Components become consistent
  - Accessibility improves
  - You stop hardcoding colors

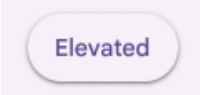
Note: if you do not provide a ThemeData, Material is activated by default but you won't have global control.

- **Advice:** Start global. Avoid styling per screen.

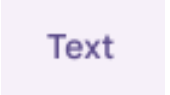
Buttons as UX Communication

- Different Buttons for different meanings

```
ElevatedButton(
  onPressed: () {},
  child: Text(”Elevated"),
)
```




```
TextButton(
  onPressed: () {},
  child: Text(”Text"),
)
```



- Button hierarchy allows to specify primary and secondary actions and put different emphasis.

- This allows also to express priorities.

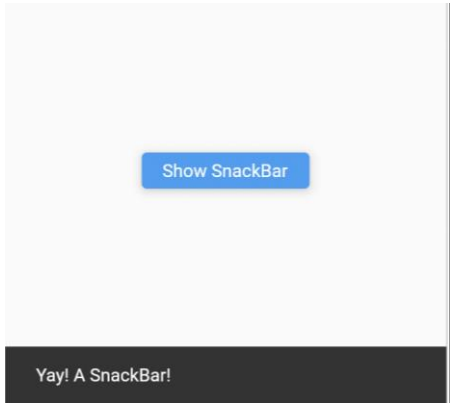
Feedback in Flutter

- Use the following to provide specific feedback.

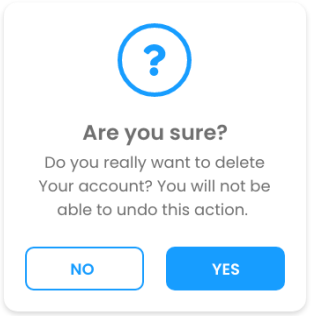
- Loading: CircularProgressIndicator()

- Confirmation:

```
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(content: Text(”Yay! A SnackBar")),
);
```



- Critical action: showDialog(…)



Defining the Structure with Scaffold

- Material screens in Flutter use the following to define the basic structure of a screen:
  - Scaffold
  - AppBar
  - NavigationBar
  - FloatingActionButton

```
Scaffold(
  appBar: AppBar(title: Text("Home")),
  body: ListView(
    padding: EdgeInsets.all(16),
    children: [
      Card(
        child: ListTile(
          title: Text(”A text"),
          subtitle: Text(”A subtitle"),
        ),
      ),
    ],
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: () {},
    child: Icon(Icons.add),
  ),
);
```

Focus on the Scaffold

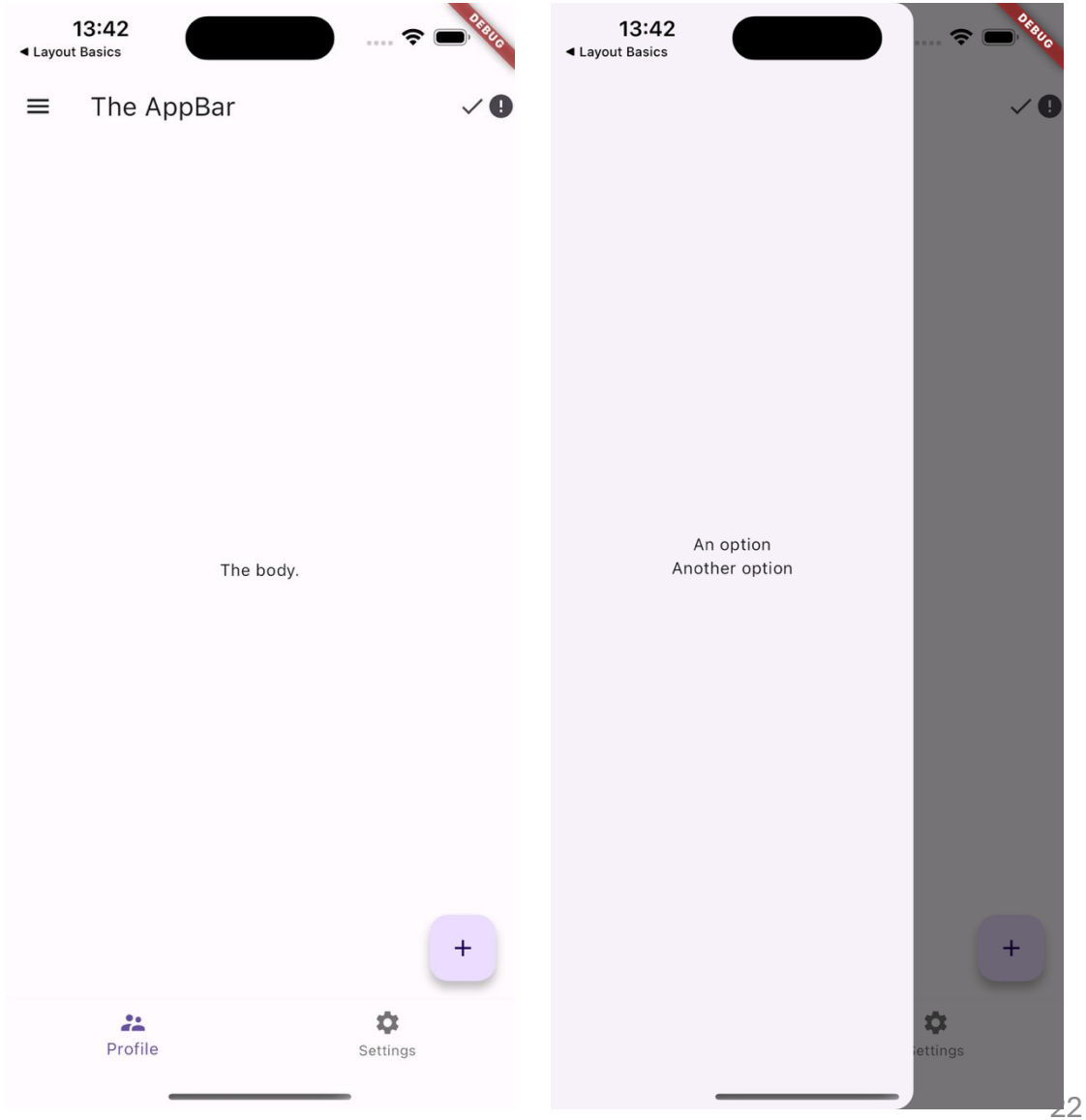
- Let’s focus into the Scaffold Widget

- Detailed info: https://api.flutter.dev/flutter/material/Scaffold-class.html

```
//Constructor of Scaffold
const Scaffold({
  Key key,
  this.appBar,
  this.body,
  this.floatingActionButton,
  this.floatingActionButtonLocation,
  this.floatingActionButtonAnimator,
  this.persistentFooterButtons,
  this.drawer,
  this.endDrawer,
  this.bottomNavigationBar,
  this.bottomSheet,
  this.backgroundColor,
  this.resizeToAvoidBottomPadding = true,
  this.primary = true,
})
```

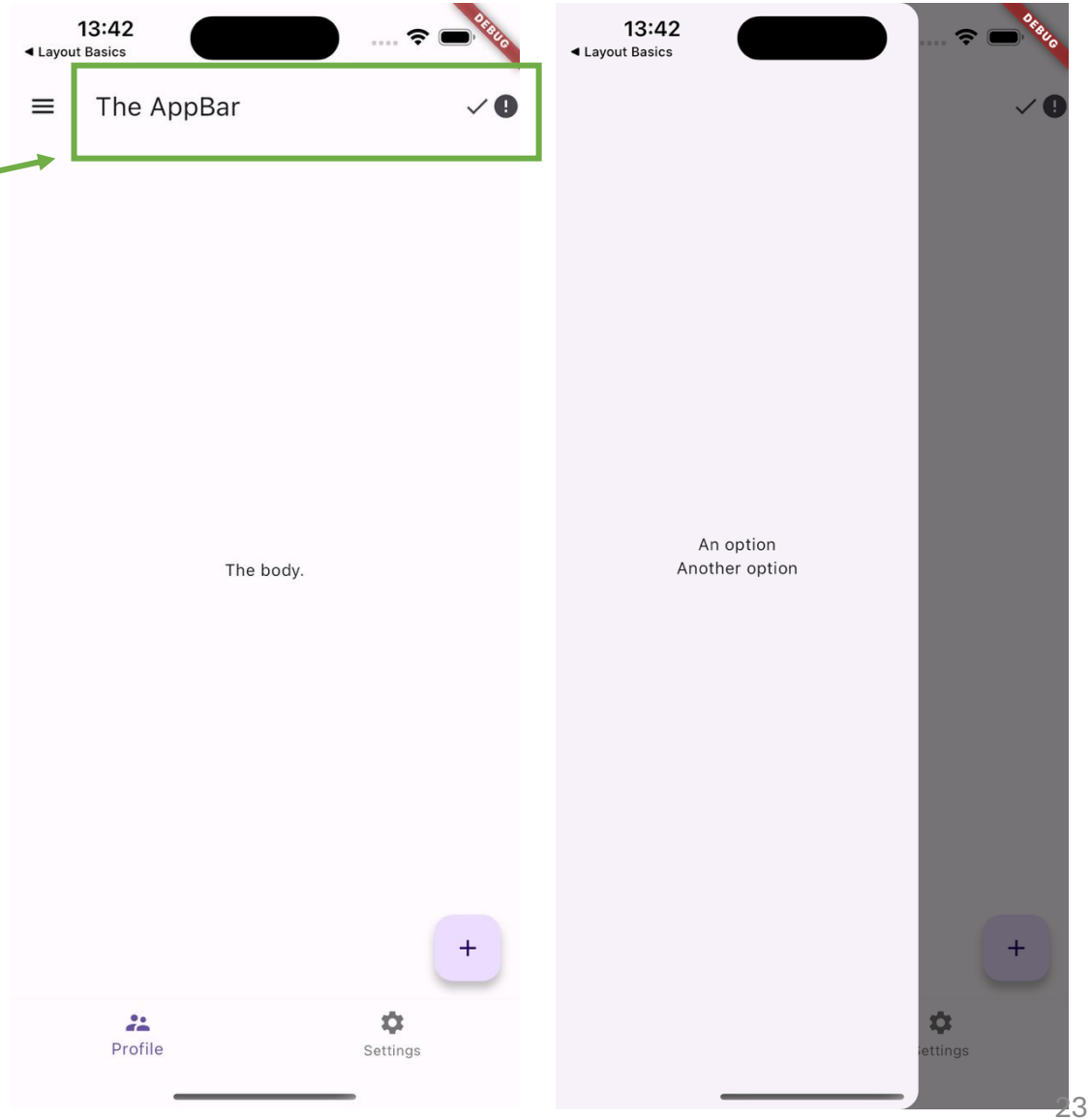
Focus on the Scaffold

```
//Constructor of Scaffold
const Scaffold({
  Key key,
  this.appBar,
  this.body,
  this.floatingActionButton,
  this.floatingActionButtonLocation,
  this.floatingActionButtonAnimator,
  this.persistentFooterButtons,
  this.drawer,
  this.endDrawer,
  this.bottomNavigationBar,
  this.bottomSheet,
  this.backgroundColor,
  this.resizeToAvoidBottomPadding = true,
  this.primary = true,
})
```



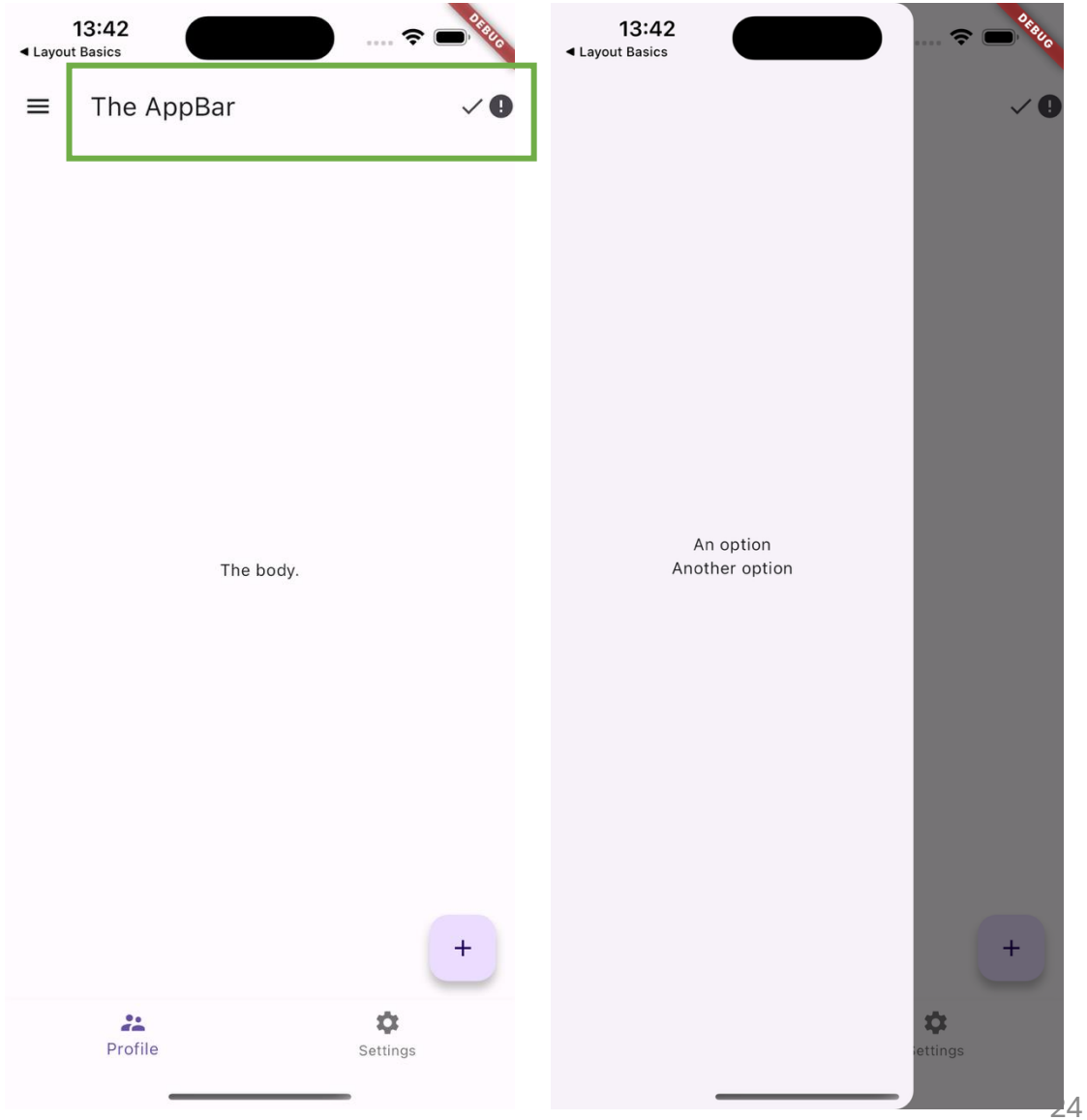
Focus on the Scaffold

```
//Constructor of Scaffold
const Scaffold({
  Key key,
  this.appBar,
  this.body,
  this.floatingActionButton,
  this.floatingActionButtonLocation,
  this.floatingActionButtonAnimator,
  this.persistentFooterButtons,
  this.drawer,
  this.endDrawer,
  this.bottomNavigationBar,
  this.bottomSheet,
  this.backgroundColor,
  this.resizeToAvoidBottomPadding = true,
  this.primary = true,
})
```



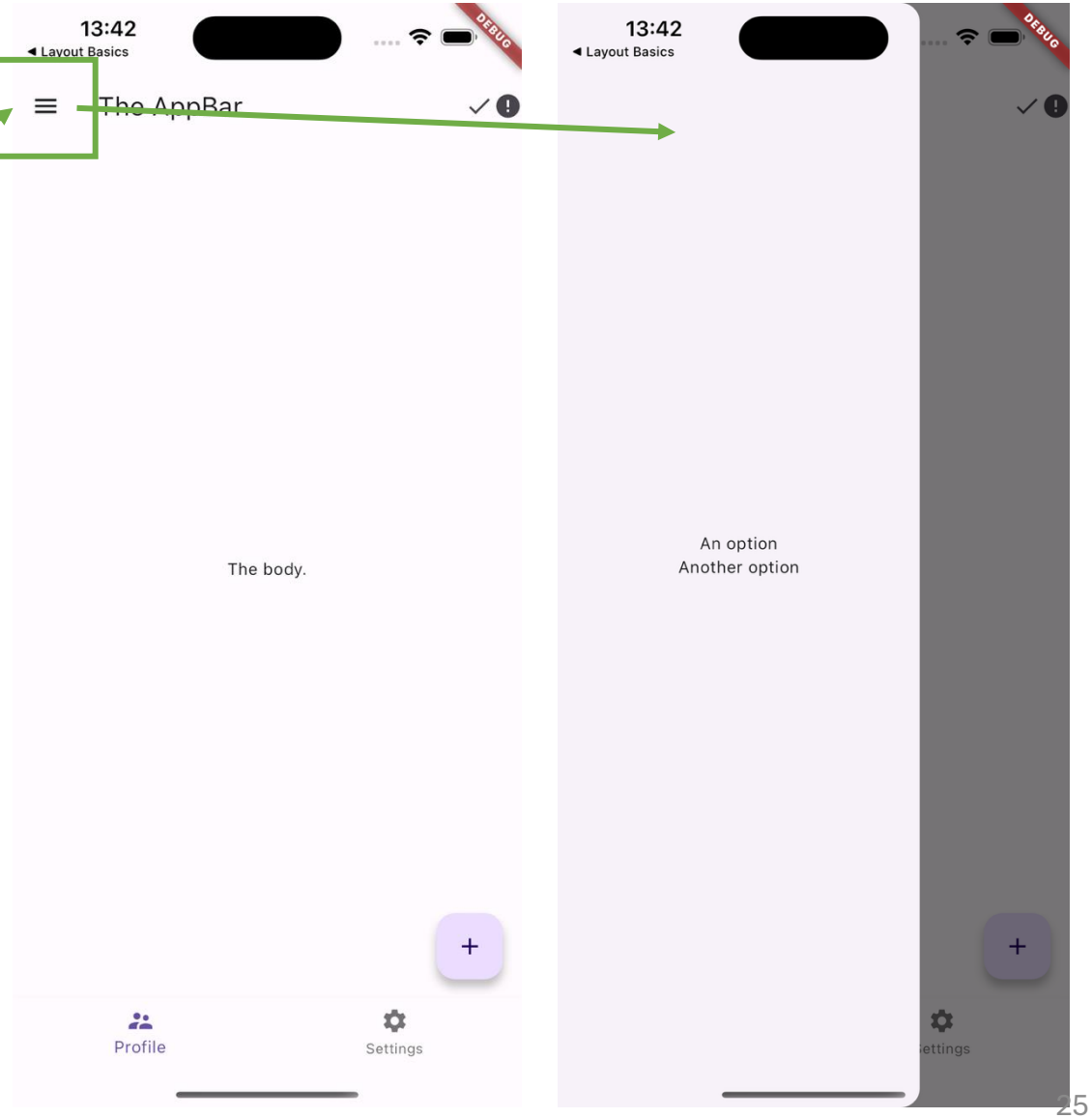
Focus on the Scaffold

```
...
home: Scaffold(
  appBar: AppBar(
    actions: [
      Icon(Icons.done,),
      Icon(Icons.error),
    ],
    title: Text('The AppBar’),
  ),
...
```



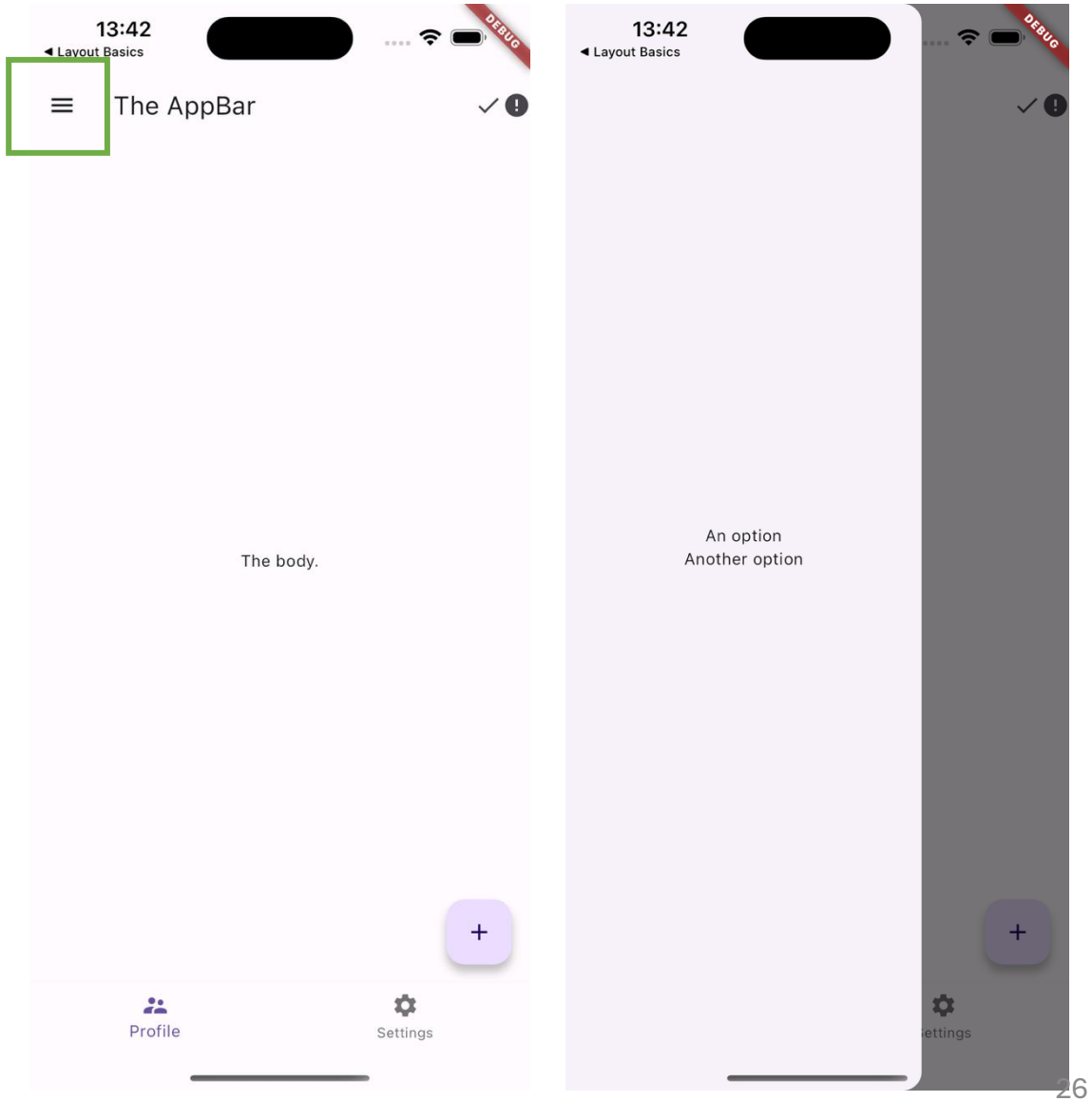
Focus on the Scaffold

```
//Constructor of Scaffold
const Scaffold({
  Key key,
  this.appBar,
  this.body,
  this.floatingActionButton,
  this.floatingActionButtonLocation,
  this.floatingActionButtonAnimator,
  this.persistentFooterButtons,
  this.drawer,
  this.endDrawer,
  this.bottomNavigationBar,
  this.bottomSheet,
  this.backgroundColor,
  this.resizeToAvoidBottomPadding = true,
  this.primary = true,
})
```



Focus on the Scaffold

```
...
home: Scaffold(
  ...
  drawer: Drawer(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text('An option’),
        Text('Another option'),
      ],
    ),
  ),
...
```

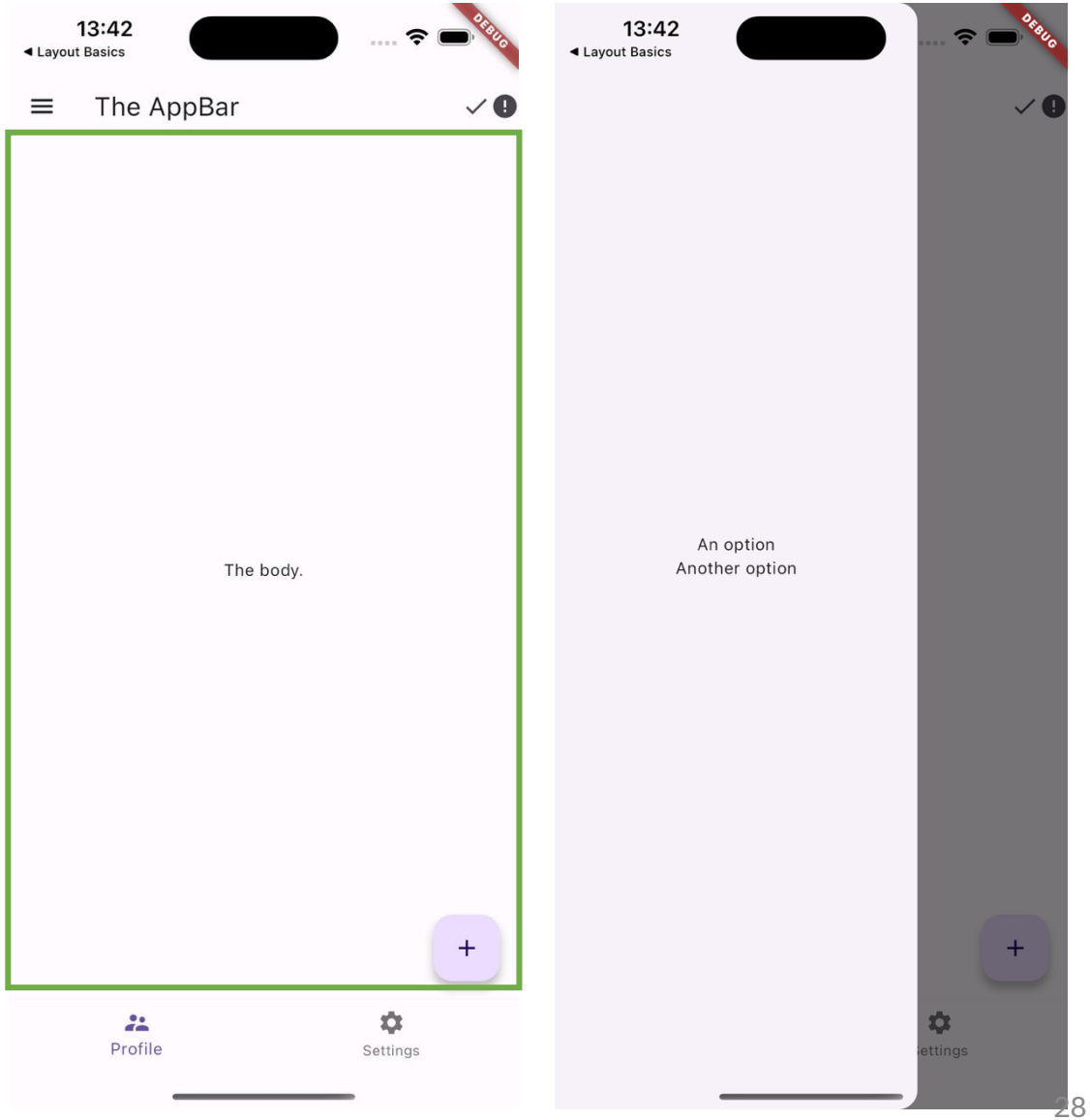


Focus on the Scaffold

```
//Constructor of Scaffold
const Scaffold({
  Key key,
  this.appBar,
  this.body,
  this.floatingActionButton,
  this.floatingActionButtonLocation,
  this.floatingActionButtonAnimator,
  this.persistentFooterButtons,
  this.drawer,
  this.endDrawer,
  this.bottomNavigationBar,
  this.bottomSheet,
  this.backgroundColor,
  this.resizeToAvoidBottomPadding = true,
  this.primary = true,
})
```

Focus on the Scaffold

```
...
home: Scaffold(
  ...
  body: Center(
    child: Text('The body.'),
  ),
...
```

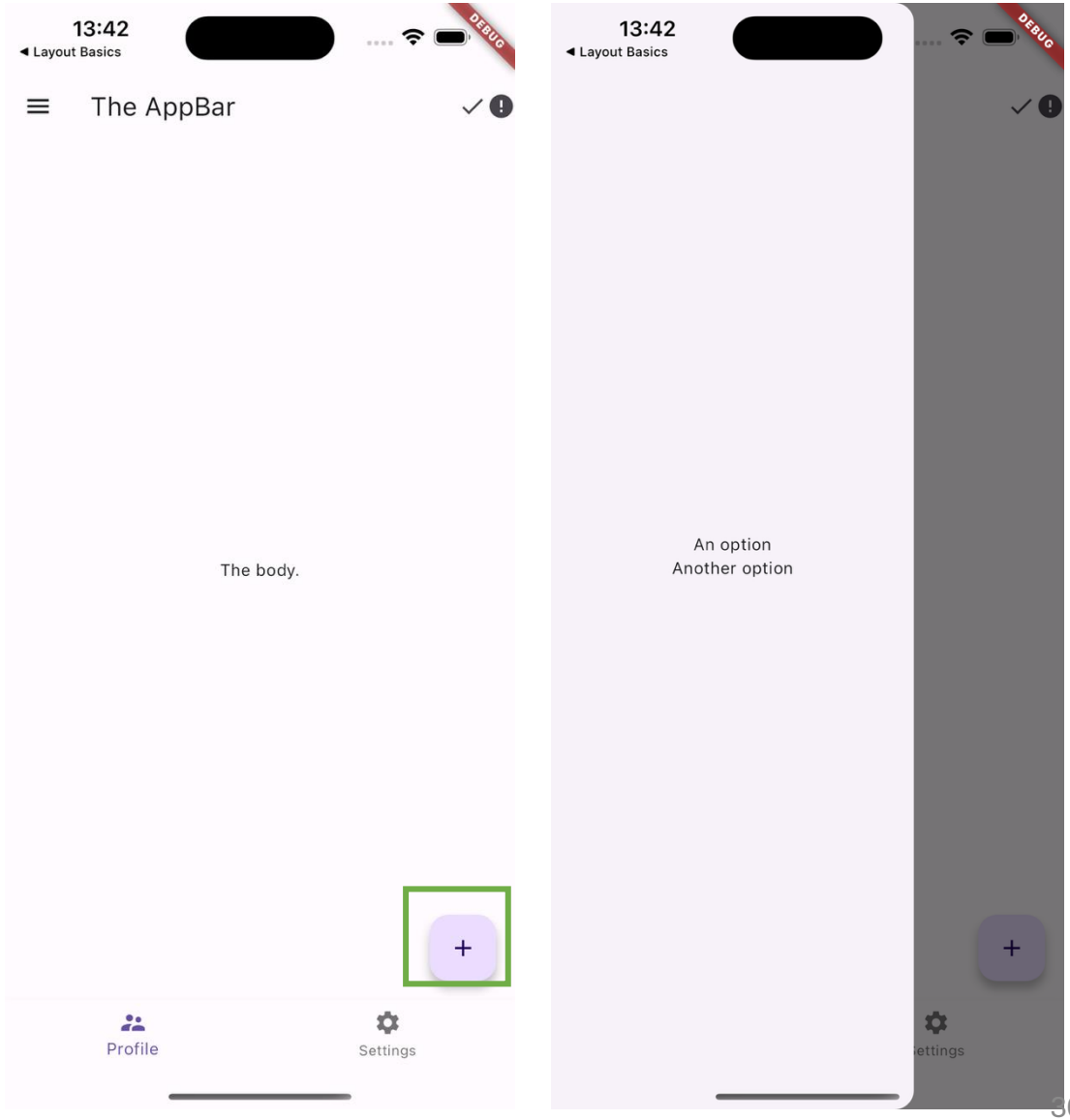


Focus on the Scaffold

```
//Constructor of Scaffold
const Scaffold({
  Key key,
  this.appBar,
  this.body,
  this.floatingActionButton,
  this.floatingActionButtonLocation,
  this.floatingActionButtonAnimator,
  this.persistentFooterButtons,
  this.drawer,
  this.endDrawer,
  this.bottomNavigationBar,
  this.bottomSheet,
  this.backgroundColor,
  this.resizeToAvoidBottomPadding = true,
  this.primary = true,
})
```

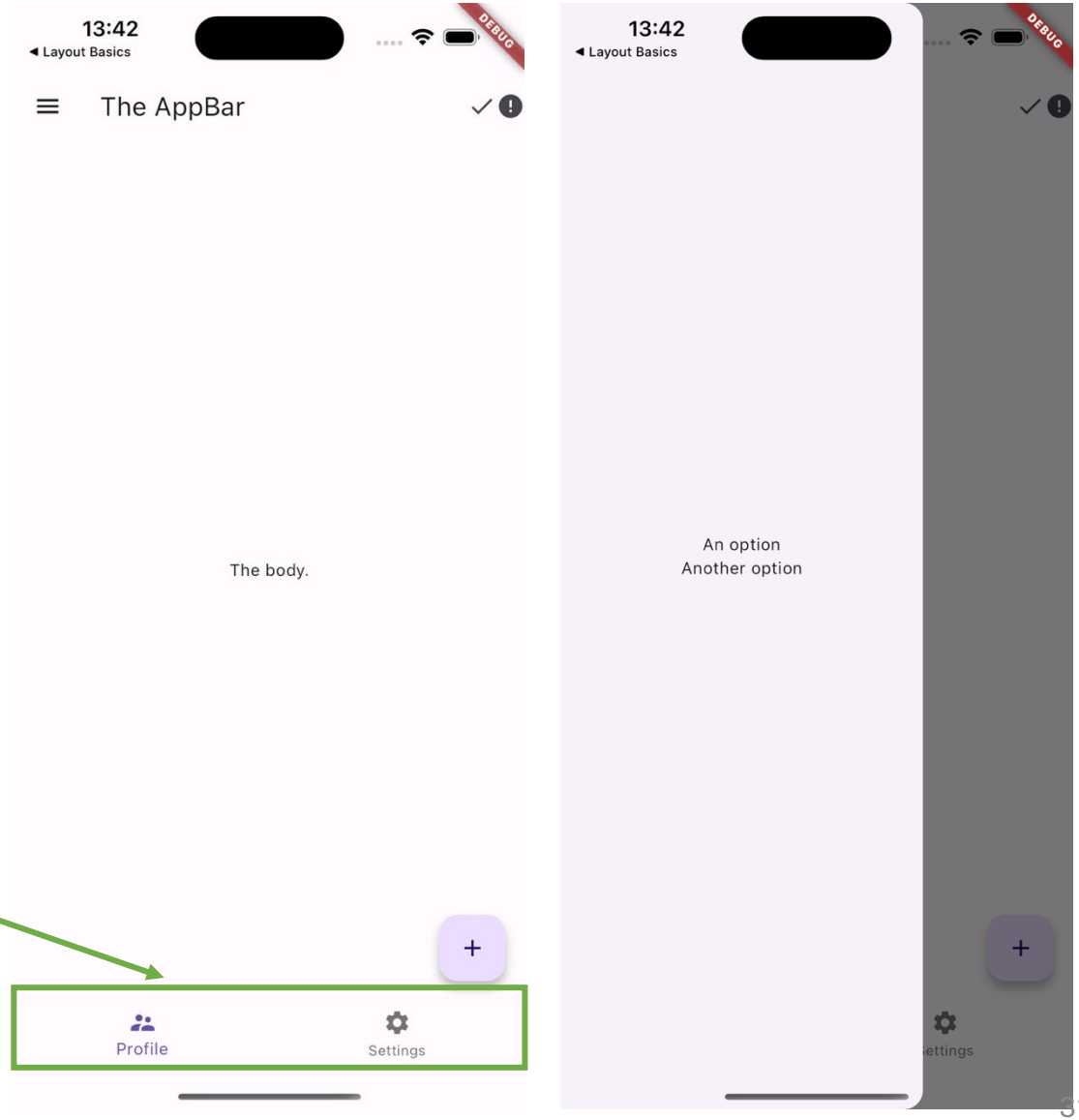
Focus on the Scaffold

```
...
home: Scaffold(
  ...
  floatingActionButton:
    FloatingActionButton(
      child: Icon(Icons.add),
      onPressed: () {},
    ),
...
```



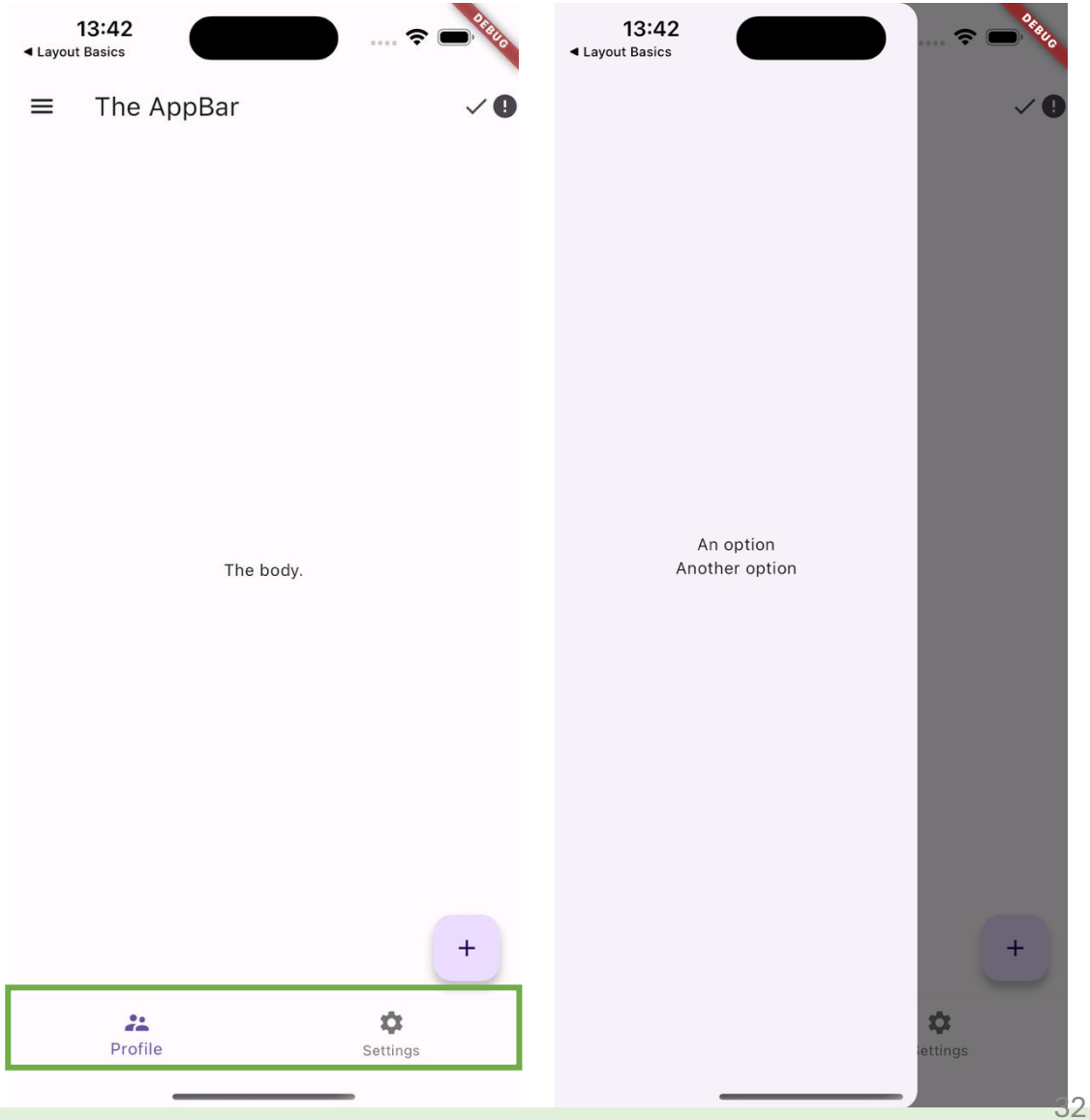
Focus on the Scaffold

```
//Constructor of Scaffold
const Scaffold({
  Key key,
  this.appBar,
  this.body,
  this.floatingActionButton,
  this.floatingActionButtonLocation,
  this.floatingActionButtonAnimator,
  this.persistentFooterButtons,
  this.drawer,
  this.endDrawer,
  this.bottomNavigationBar,
  this.bottomSheet,
  this.backgroundColor,
  this.resizeToAvoidBottomPadding = true,
  this.primary = true,
})
```



Focus on the Scaffold

```
...
home: Scaffold(
  ...
  bottomNavigationBar:
    BottomNavigationBar(
      items: [
        BottomNavigationBarItem(
          icon: Icon(Icons.supervisor_account),
          label: 'Profile',
        ),
        BottomNavigationBarItem(
          icon: Icon(Icons.settings),
          label: 'Settings',
        )
      ],
  ),
),
```



Full example in lesson_07-ux_and_ui_in_flutter/scaffolding/

Grouping Widgets together

- Grouping is how we visually communicate relationships.
- Humans automatically assume:
  - Elements close together are related.
  - Elements inside the same container belong together.
  - Aligned elements form a logical group.
- If we don’t group properly:
  - Screens feel chaotic
  - Users don’t know what belongs together
  - Cognitive load increases

- Grouping reduces mental effort.

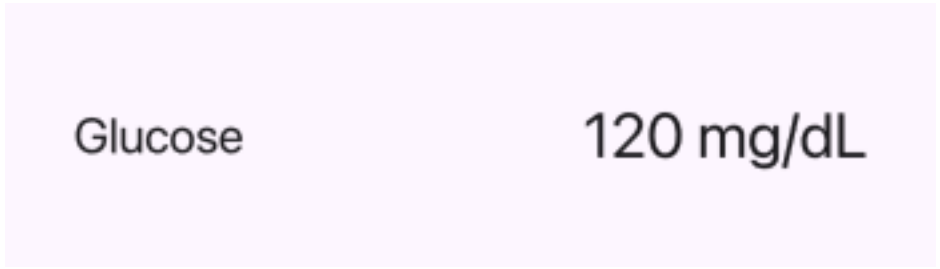
Column: The Vertical Grouping

- Use the Column Widget when:
  - Elements belong together vertically
  - The user should read top-to-bottom
  - You are structuring a section


```
Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Text(
      "Glucose",
      style: Theme.of(context).textTheme.titleLarge,
    ),
    SizedBox(height: 8),
    Text(
      "120 mg/dL",
      style: Theme.of(context).textTheme.headlineMedium,
    ),
  ],
)
```

Row: The Horizontal Grouping

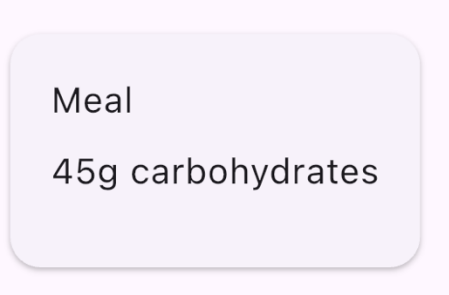
- Use the Row Widget when:
  - Items are at the same hierarchy level
  - The represent related but parallel information
  - You want side-by-side structure




```
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Text(
      "Glucose",
      style: Theme.of(context).textTheme.titleLarge,
    ),
    SizedBox(height: 8),
    Text(
      "120 mg/dL",
      style: Theme.of(context).textTheme.headlineMedium,
    ),
  ],
)
```

Card: The Unit Grouping

- Use the Card Widget to communicate that:
  - This content is a unit
  - It belongs together
  - It can be interacted with as one block




```
Card(
  child: Padding(
    padding: EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text("Meal"),
        SizedBox(height: 8),
        Text("45g carbohydrates"),
      ],
    ),
  ),
),
```

ListView

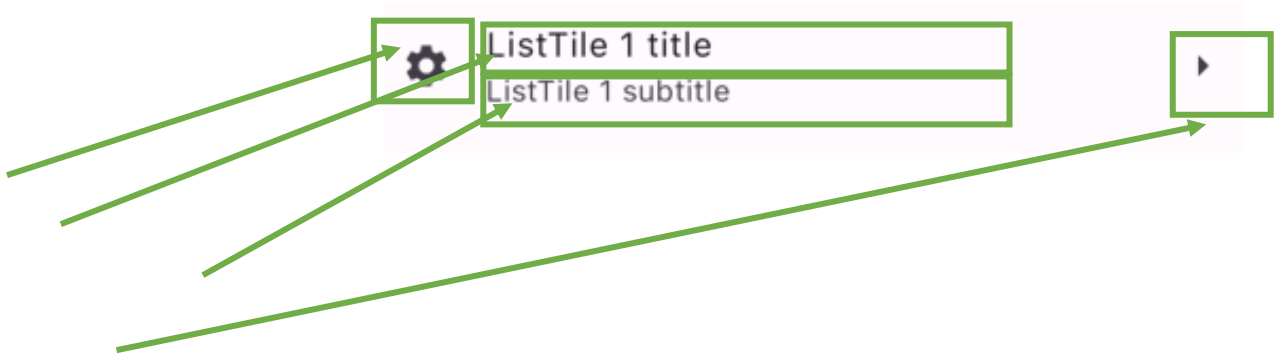
- Use ListView if you want communicate that:
  - These are similar items
  - They share the same structure
  - They can grow dynamically

- Conceptually they support scanning and pattern recognition in a UI.

- Detailed info: https://api.flutter.dev/flutter/widgets/ListView-class.html

ListView

```
...
home: Scaffold(
  ...
  body: ListView(
    children: [
      ListTile(
        leading: Icon(Icons.settings),
        title: Text('ListTile 1 title’),
        subtitle: Text('ListTile 1 subtitle’),
        trailing: Icon(Icons.arrow_right),
      ),
      ListTile(
        leading: Icon(Icons.settings),
        title: Text('ListTile 2 title’),
        subtitle: Text('ListTile 2 subtitle'),
        trailing: Icon(Icons.arrow_right),
      ),
    ],
  ),
```



Full example in lesson_07-ux_and_ui_in_flutter/list_views/